Open in app ↗

Medium

TDS Archive · Following

Member-only story

Data Science Workflows with the Targets Package in R: End-to-End Example with Code

Structured and reproducible approach for analysis

9 min read · Aug 20, 2021

Prateek Bhatnagar

Listen Share More

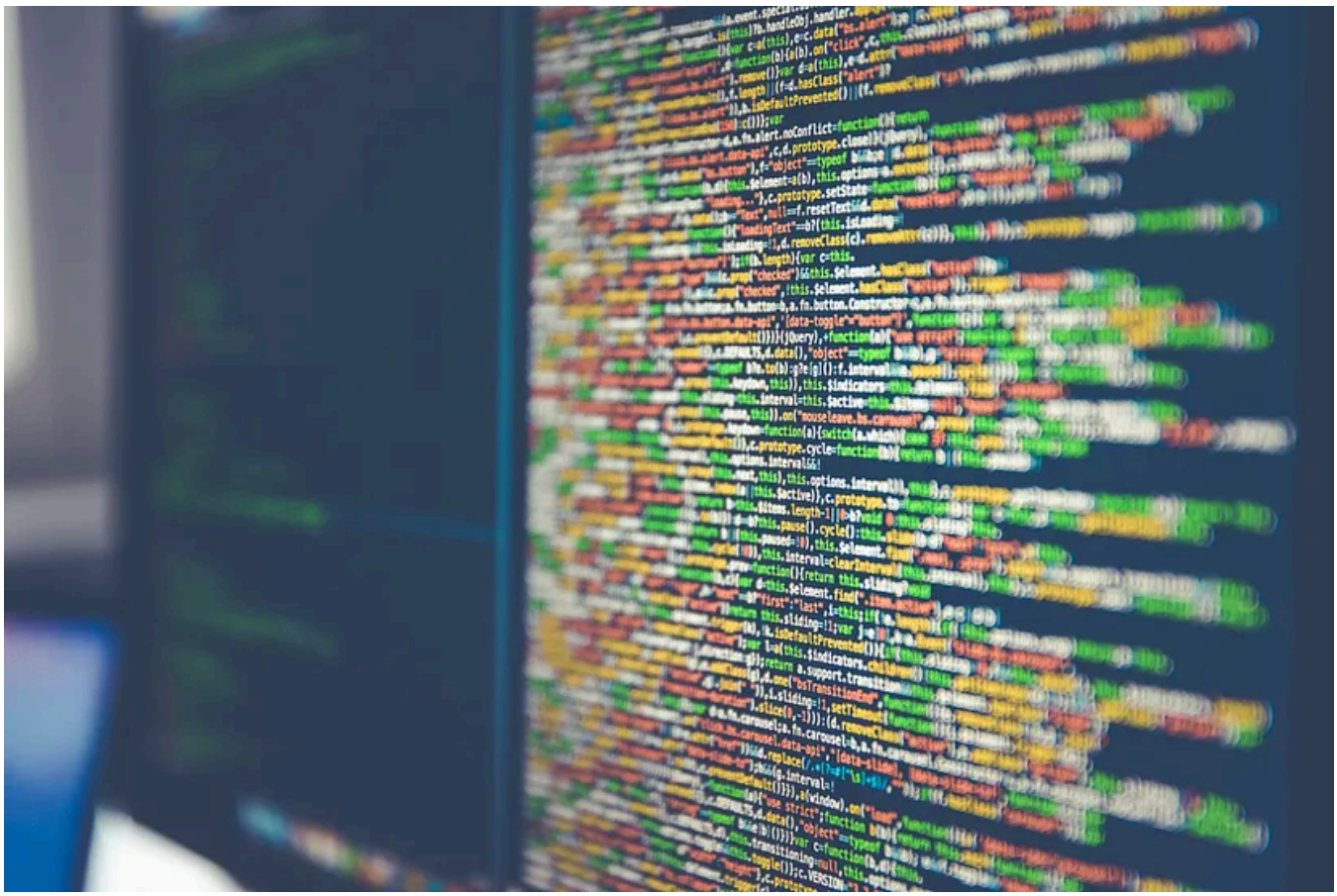


Photo by Markus Spiske on Unsplash

Introduction

Any task that we do at work or as part of our daily chores has a high likelihood of becoming repetitive in nature. This isn't to say that the task would remain exactly the same, changes may be required along the way, however structuring and organizing subtasks to accomplish the overall objective more efficiently and in a streamlined manner with minimal issues is always ideal.

Over time, we have come to adopt this "assembly line" like concept in a to our data tasks (for example, data loading, data preparation, model building, presentation of results, etc. are structured in workflows and pipelines) and the resulting improvements make a strong case for this approach. In fact, due to the iterative nature of and need of ideation in our jobs, it is imperative for data scientists to have a certain level of organization and streamlining when approaching such tasks.

Why workflows/pipelines and reproducibility is recommended?

Why do we need to organize our codes into structured workflow/pipeline format? How does it help?

1. *Data science work is computation heavy and takes time to run* — Any change along the way can imply a rerun of the whole code or parts of the code. With cost implications for both time and computation, it becomes necessary to have a streamlined approach to reruns where only updated codes and only those parts of the workflow, which are interdependent on the updated codes should run. An intelligent caching and storing function should make regeneration of results easier, efficient and error free.

2. *Easy to get lost* — Most graphs on data science workflows that you see will have a loop element. This iterative nature comes from the various parties involved, for example, domain experts, data scientists, engineers, business stakeholders, etc. Almost all data science projects go through a round of changes not only in terms of objectives but data, model, definitions, parameters, etc. which could have a downstream effect on the work done till date. In midst of all these changes and rough codes with no structure, it is very easy to get lost in the multiple updates. While tools like Github may help you to manage the code versions, it is still extremely important to put a pipeline structure to all your analysis and modularize it for better management of the overall workflow. To give an

example, in the image below, even just preparing and performing exploratory data analysis ("EDA") from the data to make a business case and recommendations on AI solution — the work can be very iterative till we get to the final result. This is illustrated from the circular loops below at almost each stage.

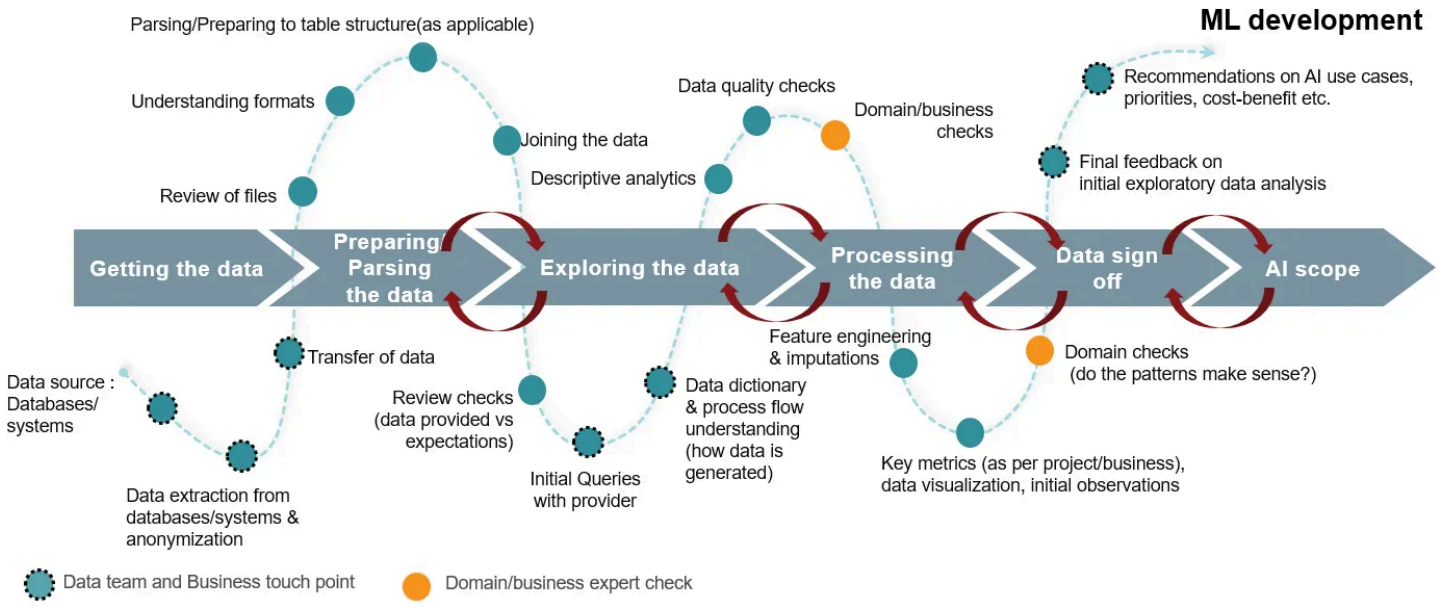


Image source : Author

3. *Clean code and Collaboration* — By turning your codes into smaller modular chunks of functions, you can break the code into small tasks, organize them in a logical order and reduce the clutter and noise from the code. This is in general a good practice. Also when working in a team and reproducing exactly the same results, such practices becomes essential for debugging, reviewing and correcting.

The Drake and Targets package

While Python has always been considered more evolved in this space, R has been catching up fast. First popular package here was Drake. It analyzes your workflow, skips steps with up-to-date results, and orchestrates the rest with optional distributed computing. At the end, `drake` provides evidence that your results match the underlying code and data, which increases your ability to trust your research.

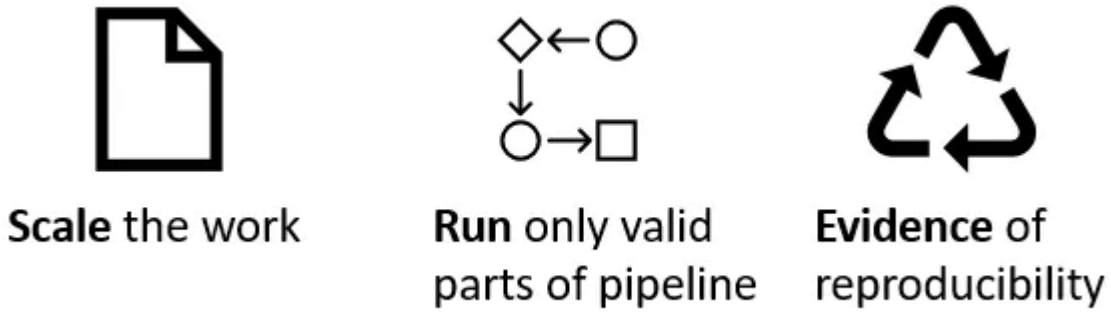


Image source : By Author

In Jan 2021, Drake was superseded by Targets, which is more robust and easier to use. It deals with a lot of gaps around data management, collaboration, dynamic branching, and parallel efficiency[1]. There are further details on enhancements and benefits of Targets over Drake which are covered here.

One of the main enhancements of Targets is the metadata management system which only updates information on global objects when the pipeline actually runs. This makes it possible to understand which specific changes to your code could have invalidated your results. In large projects with long runtimes, this feature contributes significantly to reproducibility and peace of mind[1]. A flow example given below shows parts of workflow which are up-to-date or outdated (based on user changes). This is detected real time and updated in the workflow. We will see this below in images which show a simplistic view of the workflows which can get really complex and lengthy in actual real world data science projects. Data scientists hence would appreciate the ability to quickly visualize and understand the changes, and dependencies.

Walk-through Example

In our example we will be using the popular Titanic data set. While the workflow can be as granular and complex as the user's work requires, for the purpose illustrating the utility of Targets, we will keep it simplistic and standard. Our workflow includes:

A. Loading the data
B. Pre-processing of the data
C. EDA markdown notebook generation

D. Xgboost model building and prediction on test set with some results on model diagnostics in Markdown

These steps cover the standard process data scientists follow, however there are iterations to this in each stage and we will illustrate different aspects of Targets below through this example.

1. ***Folder structure***
   Let us create a root folder called "Targets" with the below structure (this is an example, different practices may be followed by different data scientists). Within Targets we have:

Source : by Author

Important thing to note here is that `_Targets.R` should be in the root folder.

2. Creating functions which are used in targets

Here we create some functions to start with which will become a part of our workflow. As part of the example, refer to functions below which load and pre-process the data (steps A and B).

3. Defining, visualizing and executing workflow

First we create a pipeline with just tasks A and B i.e. load and pre-process the data.

Once the targets are defined, let's have a look at the flow:

```
tar_glimpse()
```

This gives a directed acyclic graph of Targets and does not account for metadata or progress information

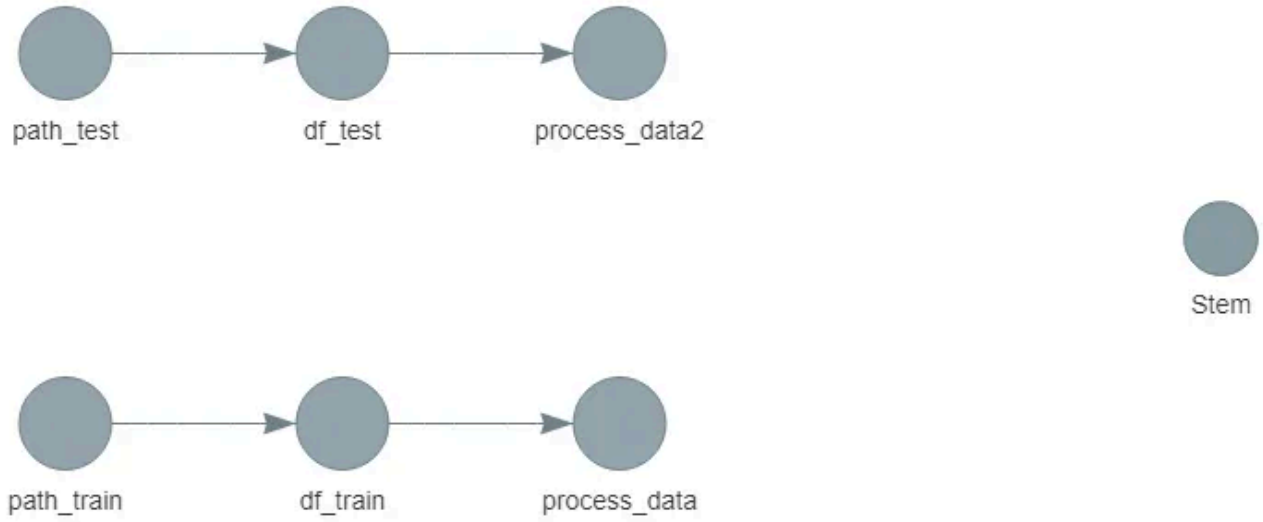


Image source : By author

```
tar_visnetwork()
```

This gives a directed acyclic graph of Targets, accounts for metadata or progress information, global functions and objects[1]. As we can see below, Targets has automatically detected the dependencies and also functions which are not used as of now anywhere for example, `bar_plot` . We also see that all of the below are outdated since we haven't run the Targets yet which will be our next step.

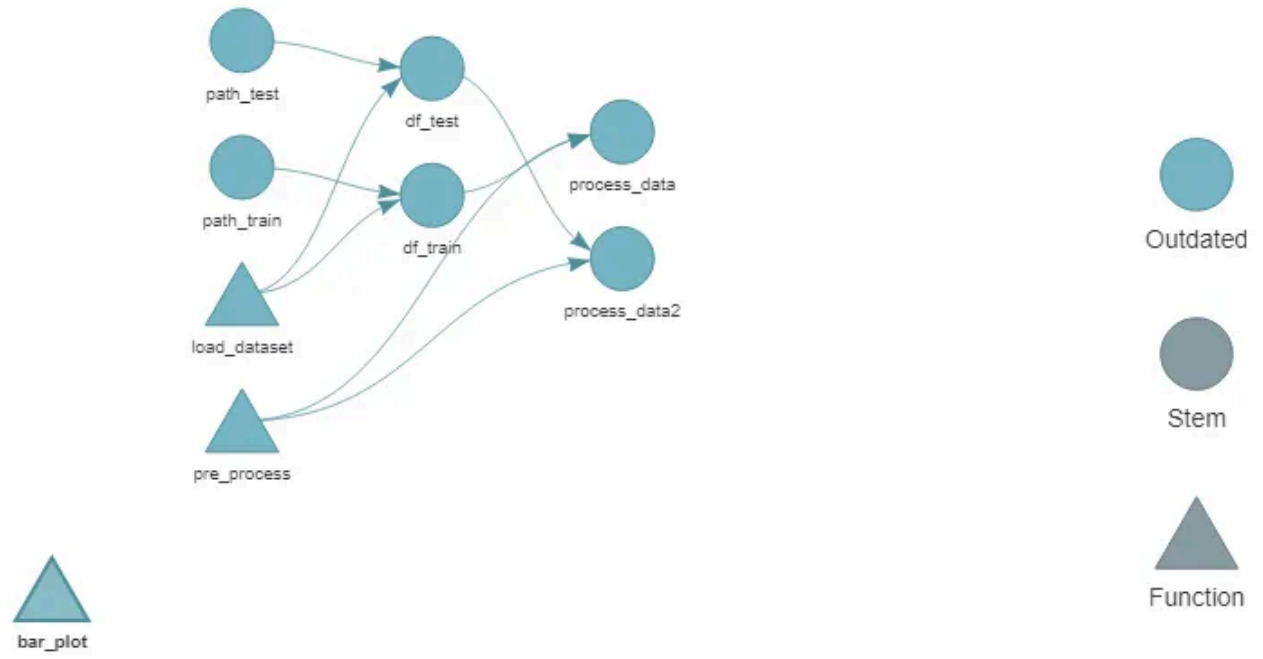


Image source : By Author

Along with the above commands, you can also use `tar_manifest()` to ensure that you have constructed your pipeline correctly.

Now we run the pipeline:

```
tar_make()
```

```
> tar_make()
* start target path_train
* built target path_train
* start target path_test
* built target path_test
* start target df_train
* built target df_train
* start target df_test
* built target df_test
* start target process_data
* built target process_data
* start target process_data2
* built target process_data2
* end pipeline
```

Image source : By author

This runs the correct targets in the correct order and stores the return values in the root folder by creating a new folder _targets. This folder will have `_targets/objects` and `_targets/meta` . Now when we visualize the pipeline using `tar_visnetwork()` , all targets have been changed to “Up to date”.

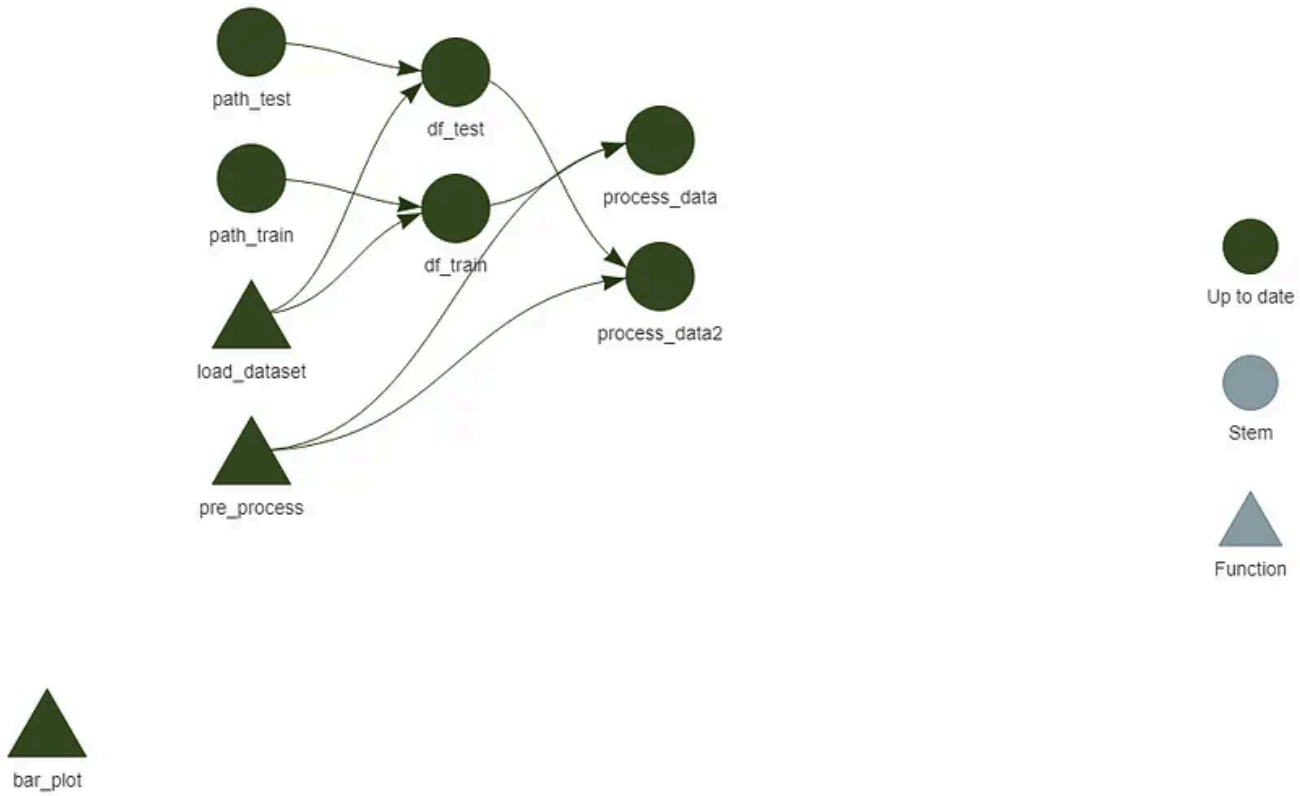


Image source : By author

4. Accessing files

To access files you can use `tar_read()` or `tar_load()` from the Targets package.

This gives us the following dataset:

```
> head(process_data_train)
   Survived Pclass                                                 Name    Sex Age SibSp Parch
1:        0      3                              Braund, Mr. Owen Harris   male  22     1     0
2:        1      1 Cumings, Mrs. John Bradley (Florence Briggs Thayer) female  38     1     0
3:        1      3                               Heikkinen, Miss. Laina female  26     0     0
4:        1      1       Futrelle, Mrs. Jacques Heath (Lily May Peel) female  35     1     0
5:        0      3                             Allen, Mr. William Henry   male  35     0     0
6:        0      1                              McCarthy, Mr. Timothy J   male  54     0     0
            Ticket    Fare Cabin Embarked
1:        A/5 21171  7.2500              S
2:         PC 17599 71.2833   C85        C
3: STON/O2. 3101282  7.9250              S
4:           113803 53.1000  C123        S
5:           373450  8.0500              S
6:            17463 51.8625   E46        S
```

Image source : By Author

As mentioned in the previous section the files are also stored in the objects section:

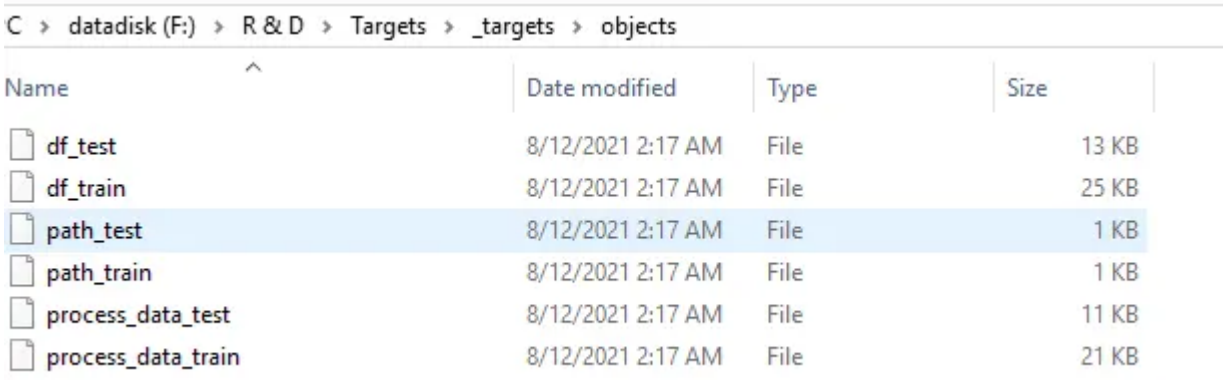


Image source : By Author

You can also specify the format you want to store the files in e.g. `rds` . Targets can return multiple files, which can be stored as a list and returned in the target and then retrieved as `target_loaded_file@.`.

5. Changes to workflow

I. Changes within same workflow

First let us start by making a change in the overall workflow. For example, making a change in the load dataset function:

Once we have made this update and checked the workflow via `tar_visnetwork()` we can see below that Targets automatically detects the dependency and outdates all the following targets accordingly. In `tar_make()` it will rerun all the outdated targets accordingly.

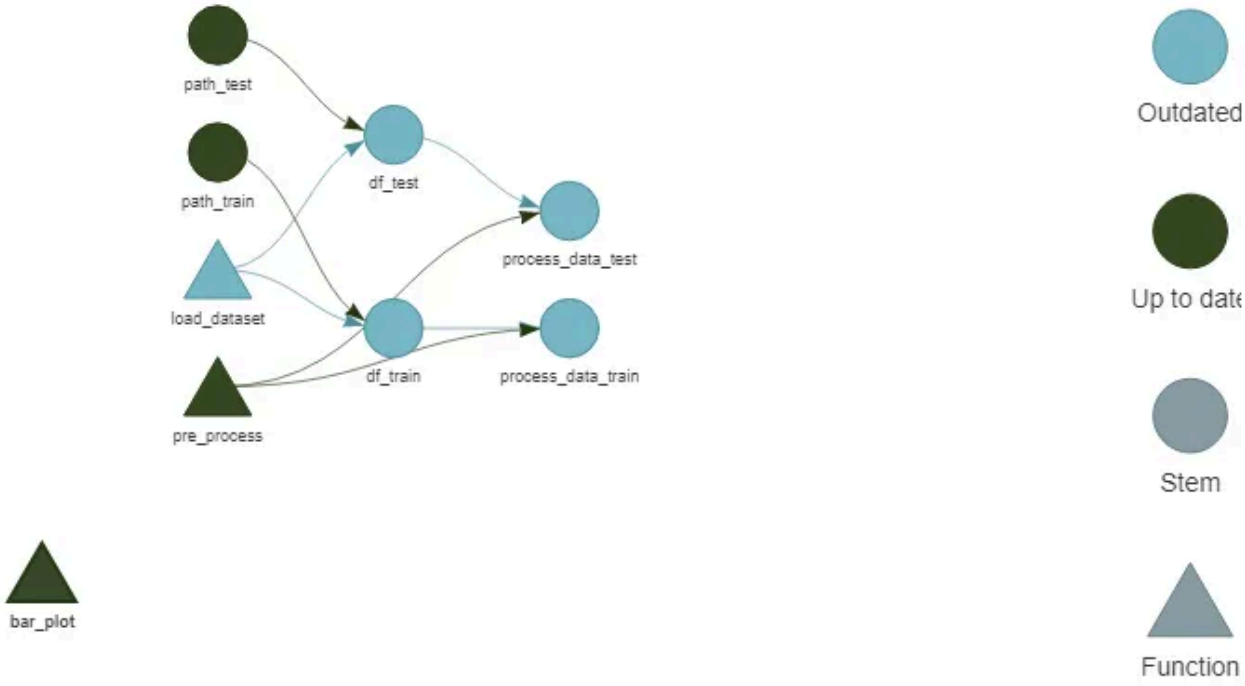


Image source : by Author

Similarly, for example if we make a change in Step B, which is process data code, and remove `na.omit()`

and then review the workflow:

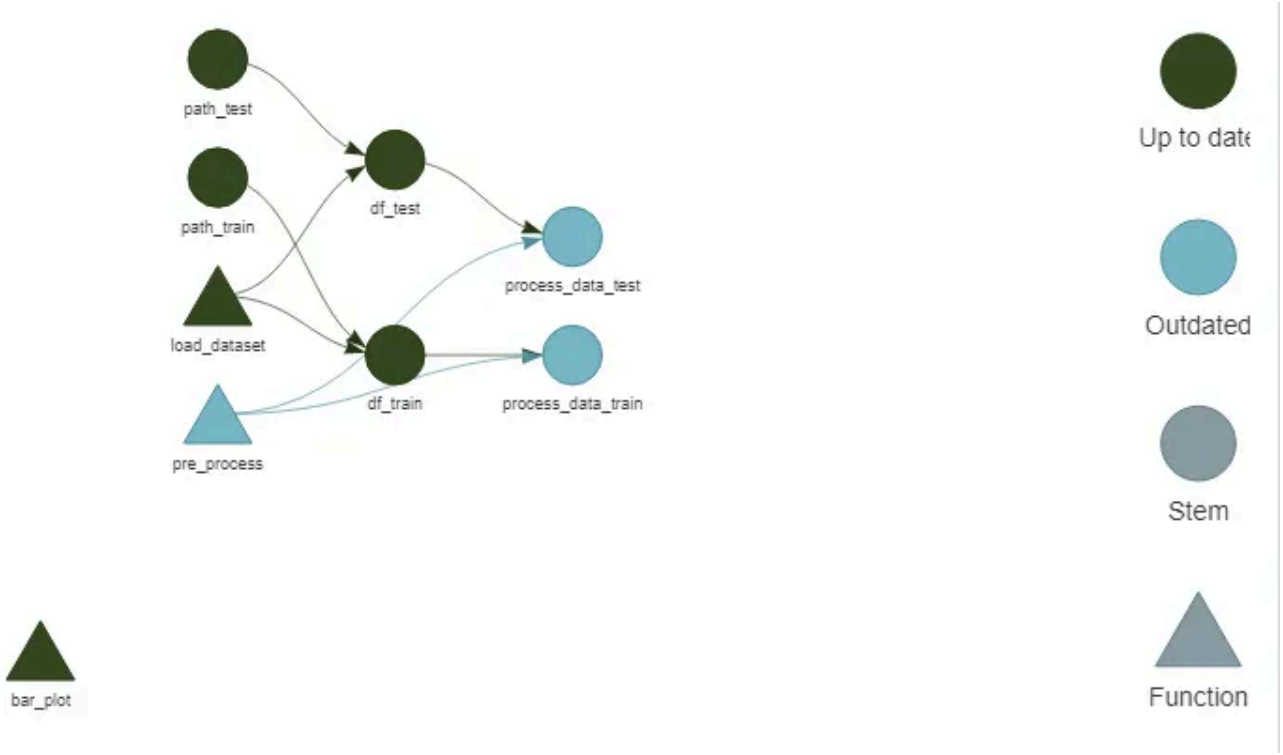


Image source : By Author

we can see only “process_data_train” and “process_data_test” are outdated and need to be rerun. Hence Targets will skip the previous components in `tar_make()`.

```
> tar_make()
v skip target path_train
v skip target path_test
v skip target df_train
v skip target df_test
* start target process_data_train
* built target process_data_train
* start target process_data_test
* built target process_data_test
* end pipeline
```

Image source : By Author

II. Addition to workflow: EDA

Now let us add a short EDA markdown notebook (Step C) to the workflow. Below is a sample code:

This can be added in `_Targets.R` as below:

This is reflected in the new process flow:

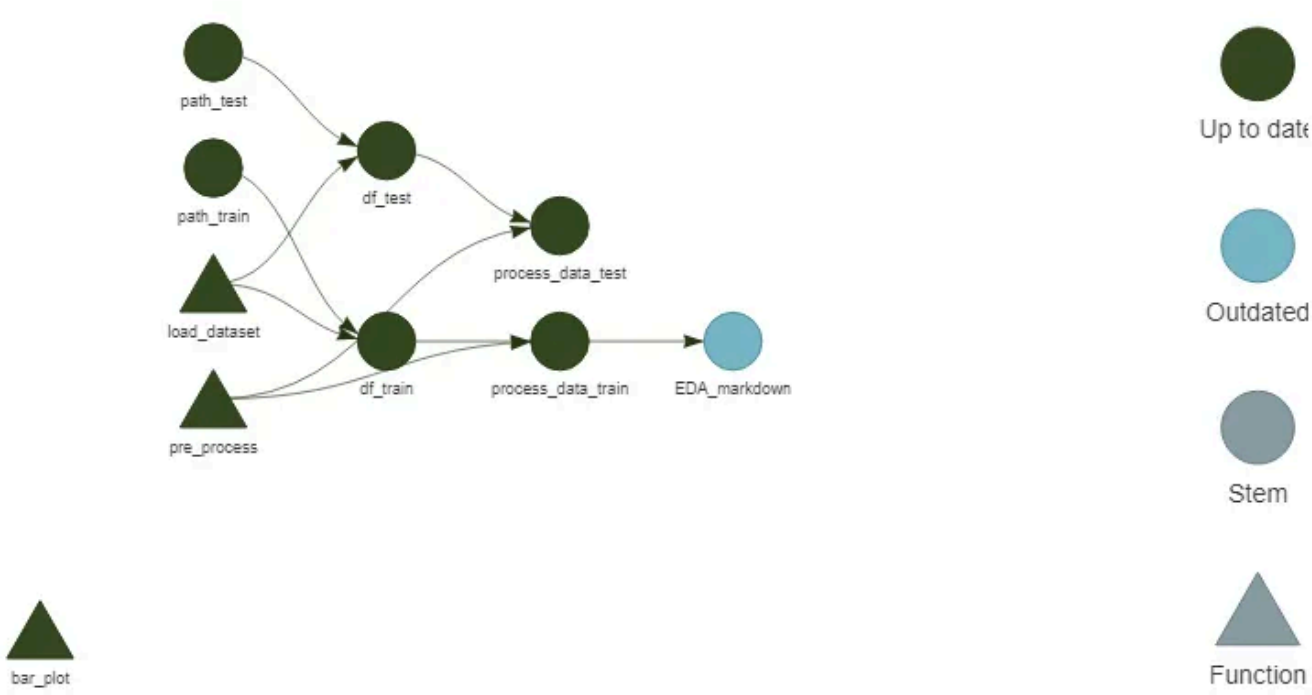


Image source : By Author

This gives us the EDA markdown for train data set. This can easily be reused for test data set as well by simply changing the source target data loaded in the code and creating a new target in `_Targets.R` .

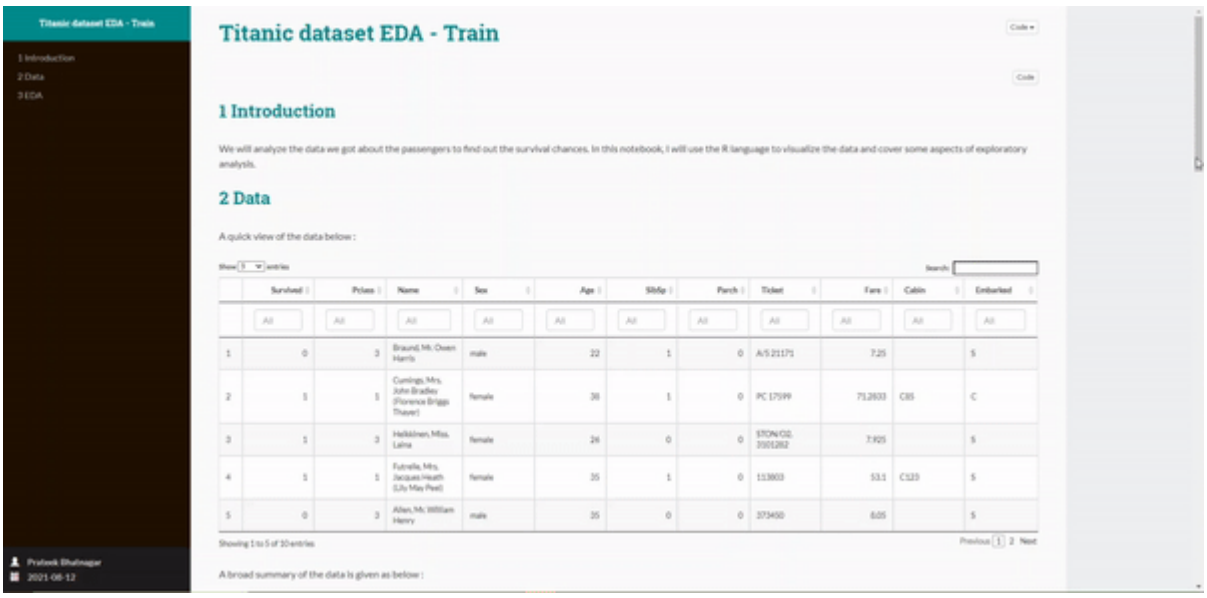


Image source : By Author

III. Addition to workflow : Modelling markdown and prediction

Similar to EDA markdown, we can now create a markdown to create a model(can also be put into a separate function) and show our model results. A sample `Rmd` is given below (builds the model, generates model diagnostics visuals, makes prediction on test and saves results) followed by new process flow:

We add this to the targets flow.

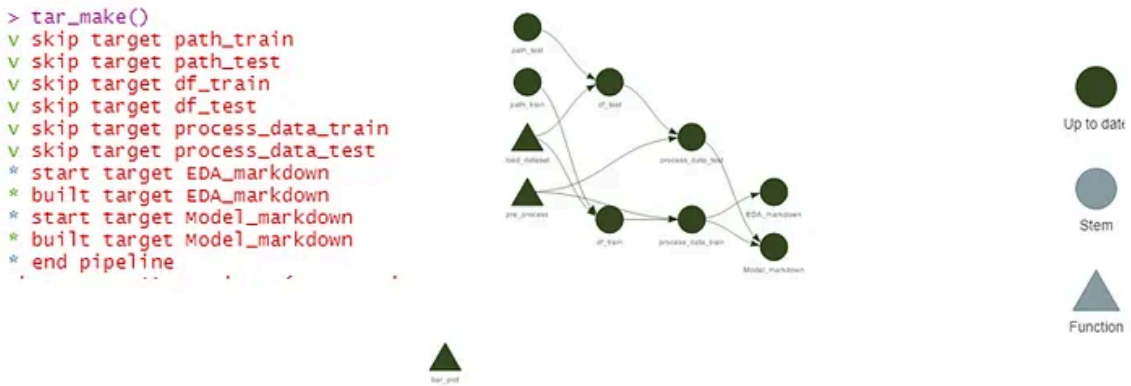


Image source : By Author

The resulting markdown looks like this:

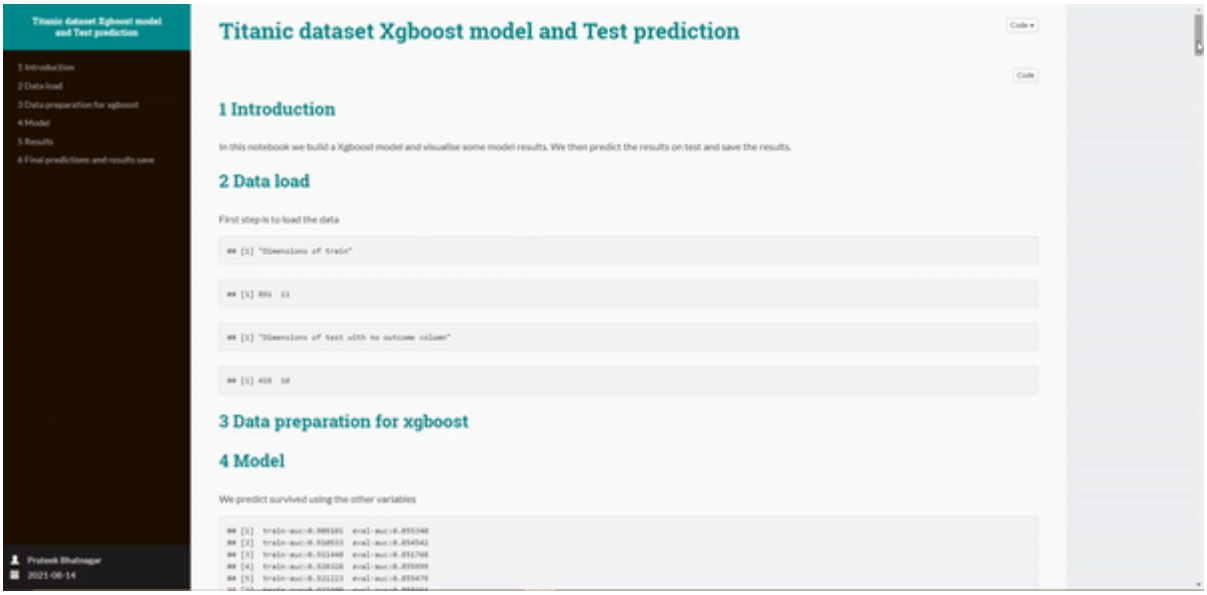


Image source : by Author

There are many other functionalities which can help in ordering, prioritizing, destroying targets and many more can be found at Targets page.

A note on renv & Docker — towards complete reproducibility and productionizing
In ensuring your work is finally production ready, a few more components get involved and it is always best to have working knowledge of these to master reproducibility in development or deployment phase.

1. Renv brings project-local R dependency management which enables your colleagues and reviewers to recreate the same environment as your development to produce your results easily. R users can appreciate this given the many versions and updates that come through for various packages. A brilliant blog tutorial by my colleague Liu Chaoran can be accessed here which provides details on how to use renv.

2. Docker is the final step and works beautifully with renv. Previously when we want to containerize R code with docker for production/deployment, we needed to create a separate R code which lists all the `install.packages` commands. Now we can conveniently call one line of code[4] using renv. The article by my colleague above covers Docker as well.

Having pipeline-like toolkit such as Targets together with dependency and containerizing toolkits such as renv and Docker — R as a language is progressing fast in the production ready deployment space.

Some other options and references:

There are some other (though not many) resources on Targets but there are more on Drake which is a much older package. On Targets, some good videos by the author can be accessed here, and another good watch is this video by Bruno.

Apart from Targets there are some other packages that can be explored such as Remake package. Although appearing to be very similar to Targets, I have not used this package yet to comment.

Another option is Ruigi, which is similar to its counterpart in Python Luigi. Python users, jumping to R may prefer this package.

Finally, for a finely curated list of pipeline toolkits across different languages and aspects of work refer to:

GitHub - pditommaso/awesome-pipeline: A curated list of awesome pipeline toolkits inspired by...

A curated list of awesome pipeline toolkits inspired by Awesome Sysadmin DVC - Data version control system for ML...

github.com

Concluding note

While the above example is rather simplistic and may not be reflective of the complexities, iterations and range of outputs that data science work may have — it reflects the utility of Targets and only goes out to show how useful it could be as work gets more complex and codes get more messy.

References

1. https://books.ropensci.org/targets/
2. https://towardsdatascience.com/this-is-what-the-ultimate-r-data-analysis-workflow-looks-like-8e7139ee708d

3. https://rstudio.github.io/renv/articles/renv.html
4. https://6chaoran.github.io/data-story/data-engineering/introduction-of-renv/
5. https://swcarpentry.github.io/r-novice-inflammation/02-func-R/

Data Science | Workflow | Reproducibility | Pipeline As Code | R

Data Science

Following

Published in TDS Archive

828K followers · Last published Feb 4, 2025

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.

Edit profile

Written by Prateek Bhatnagar

175 followers · 43 following

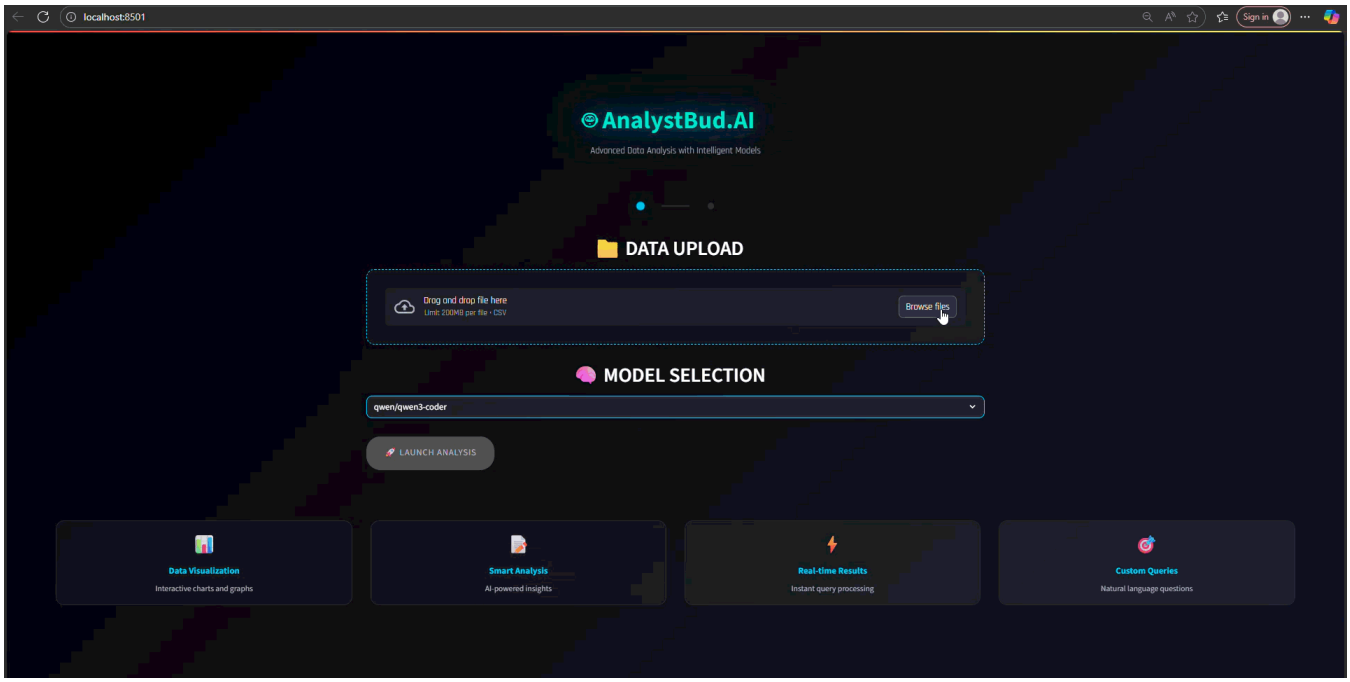
Data Geek, Short story writer

No responses yet

Prateek Bhatnagar

What are your thoughts?

More from Prateek Bhatnagar and TDS Archive

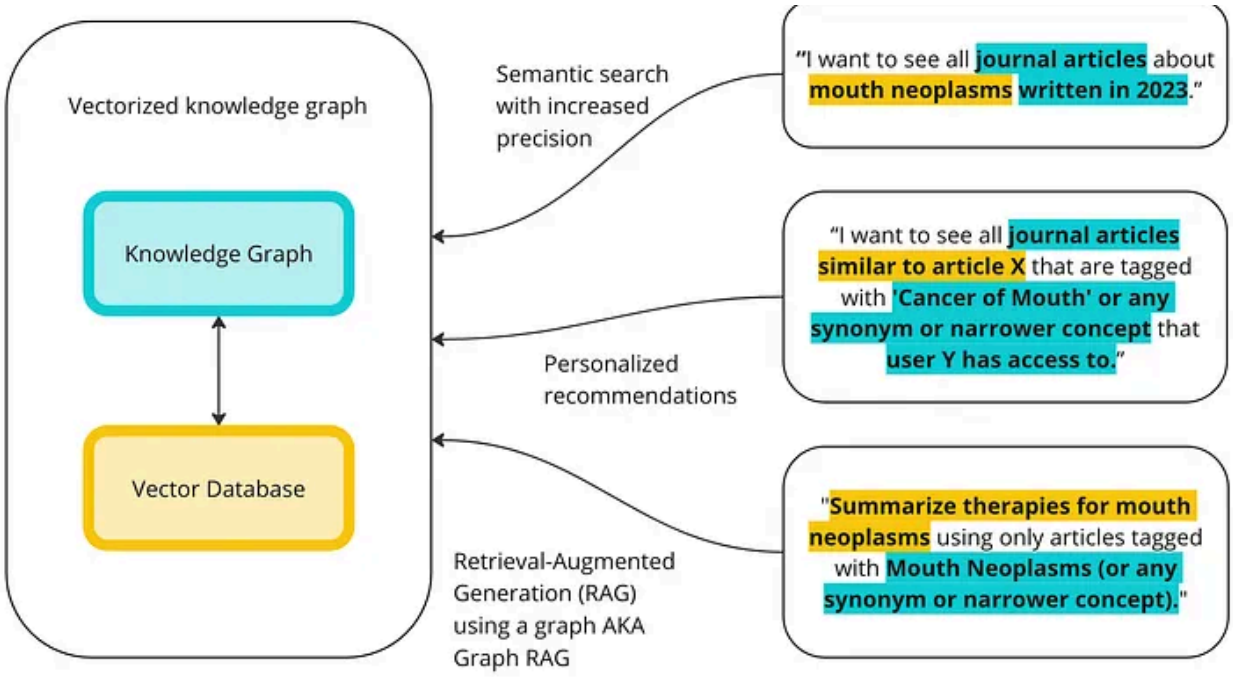


In Data Science Collective by Prateek Bhatnagar

Building conversational analytics tool with Langchain and Qwen3 Coder : Hands on guide with lessons

Learnings from a weekend project to create an intelligent data analysis agent that can understand natural language queries and generate...

Sep 7, 2025 54

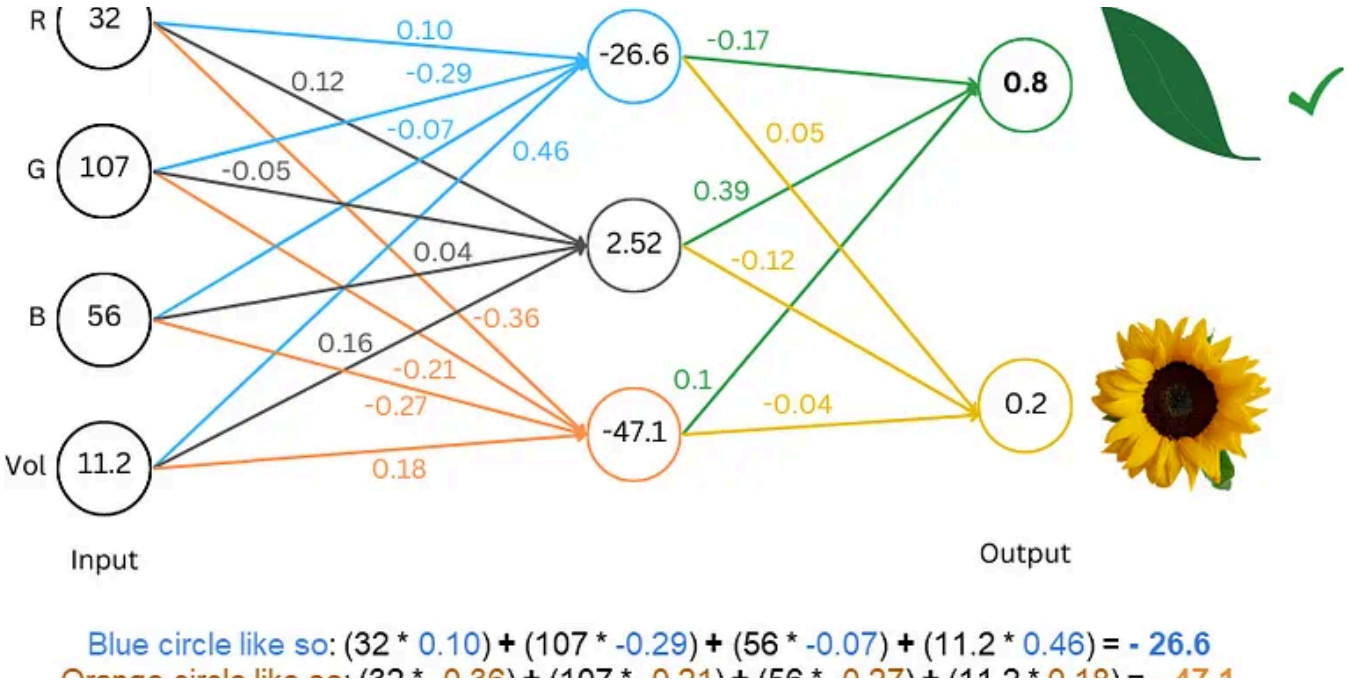


In TDS Archive by Steve Hedden

How to Implement Graph RAG Using Knowledge Graphs and Vector Databases

A Step-by-Step Tutorial on Implementing Retrieval-Augmented Generation (RAG), Semantic Search, and Recommendations

Sep 7, 2024 2K 20



In TDS Archive by Rohit Patel

Understanding LLMs from Scratch Using Middle School Math

In this article, we talk about how LLMs work, from scratch—assuming only that you know how to add and multiply two numbers. The article...

Oct 20, 2024 8.3K 105

Prateek Bhatnagar

The empty house

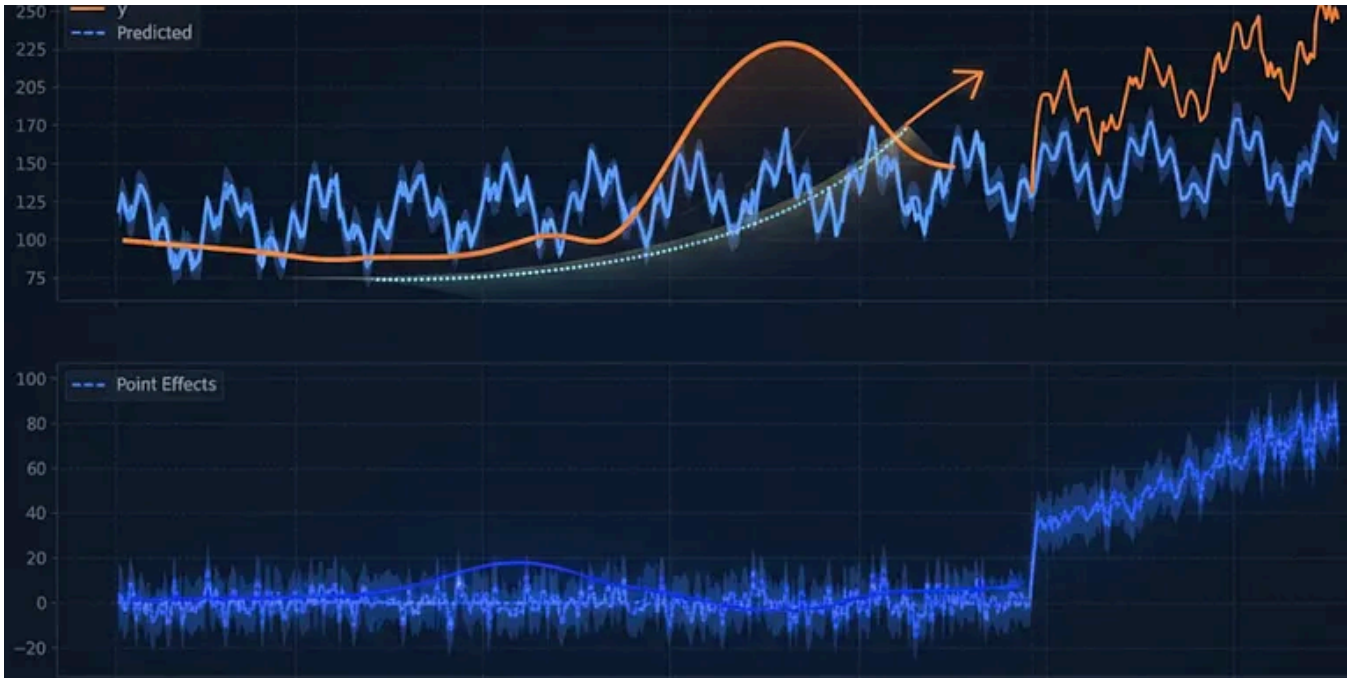
I couldn't ignore my thoughts anymore. It was no longer the same when I would open the door and walk into an aura of togetherness ...

Apr 16, 2021 14

See all from Prateek Bhatnagar

See all from TDS Archive

Recommended from Medium

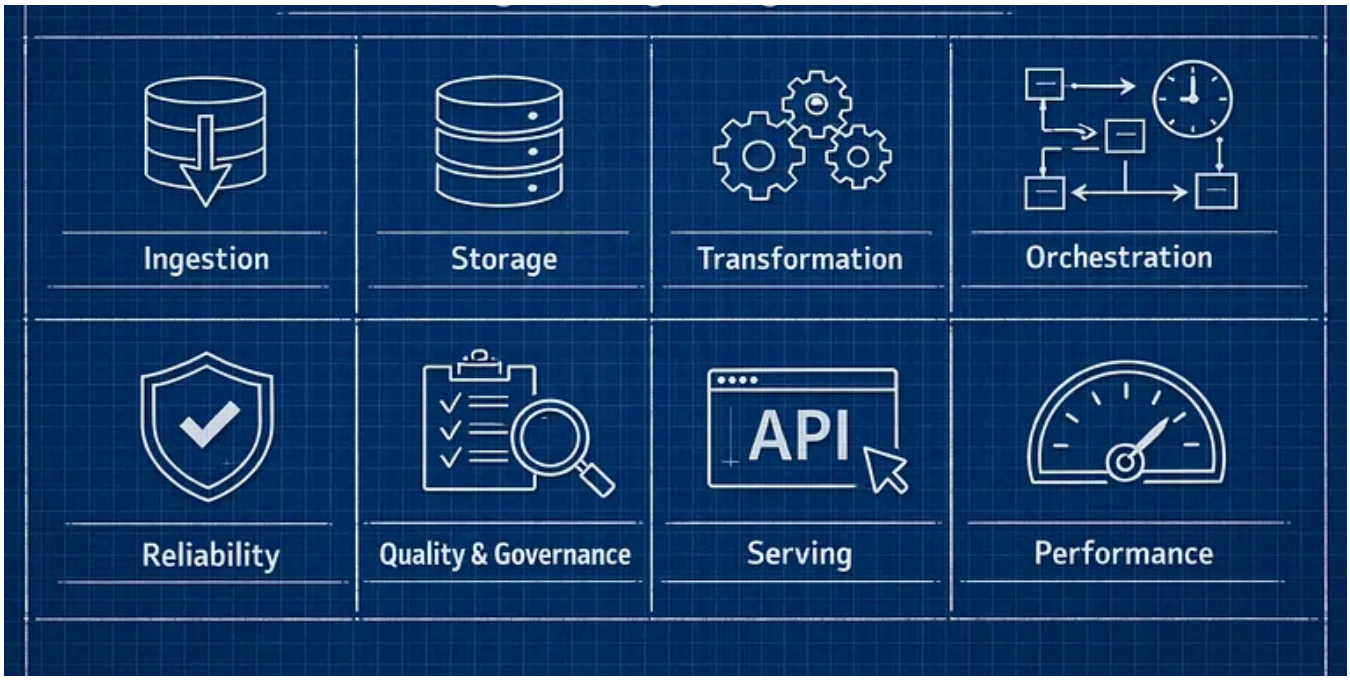


In Data Science Collective by Sophia Chen, PhD

Causal Inference Application v3: Introducing Bayesian Structural Time Series (BSTS)

Evaluating Intertemporal Causal Impact

Feb 16 65

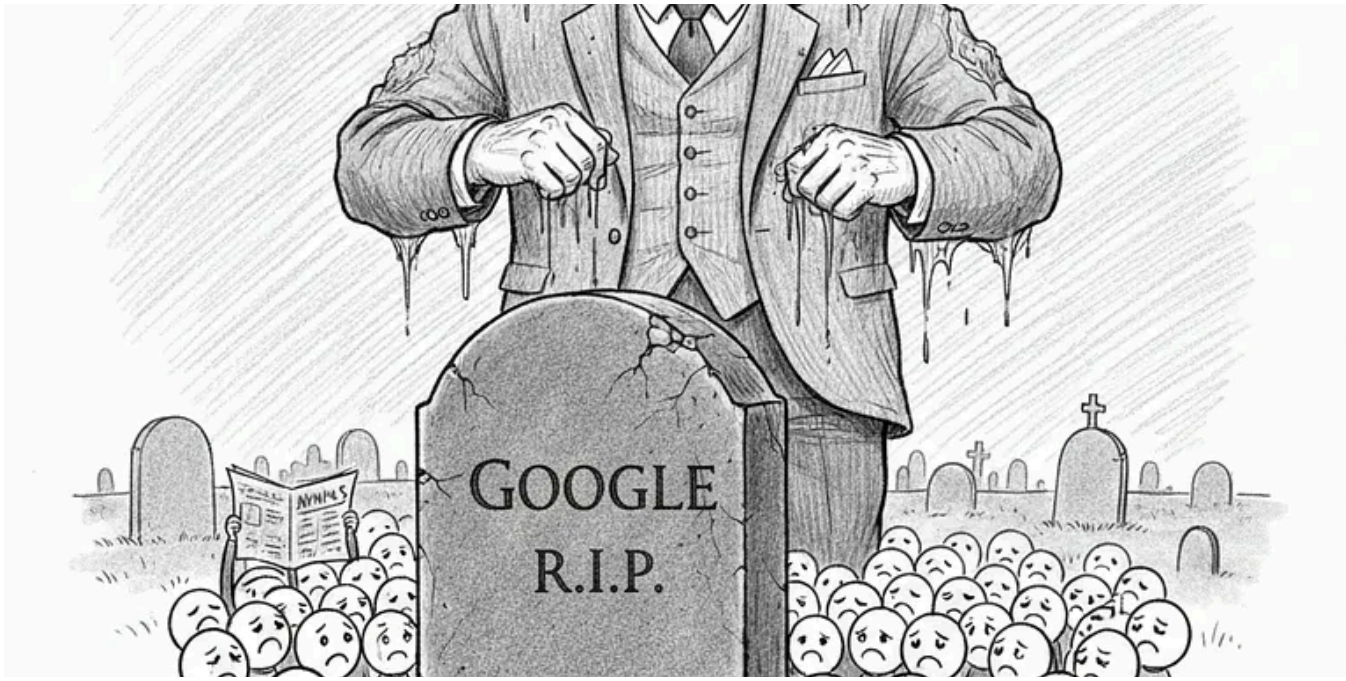


In AWS in Plain English by Khushbu Shah

Data Engineering Design Patterns You Must Learn in 2026

These are the 8 data engineering design patterns every modern data stack is built on. Learn them once, and every data engineering tool…

Jan 5 1.1K 25

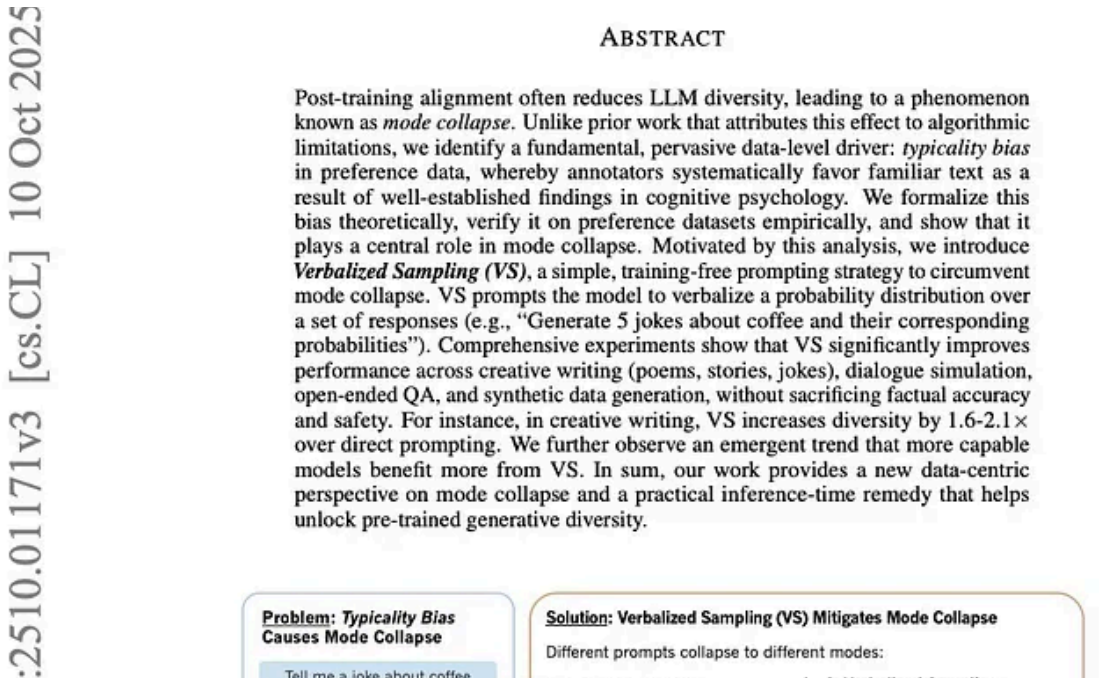


In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn't bankruptcy. It's becoming the engine inside someone else's car.

Feb 18 3.7K 127



:2510.01171v3 [cs.CL] 10 Oct 2025

ABSTRACT

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce ***Verbalized Sampling (VS)***, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., "Generate 5 jokes about coffee and their corresponding probabilities"). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.

Problem: *Typicality Bias* Causes Mode Collapse

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

Different prompts collapse to different modes:

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model—no training required…

Oct 20, 2025 24K 643

In Artificial Corner by The PyCoach

The Best AI Tools for 2026

If you're going to learn a new AI tool, make sure it's one of these

Dec 1, 2025 4.8K 272

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big Tech AI Subscriptions Forever |...

Something strange happened in early 2026. Apple stores started running low on Mac Minis. Tech forums exploded with setup guides. Developers...

Feb 15 3.1K 50

See more recommendations